Document-Grounded Conversational AI using RAG

Objective

Build a **working conversational agent** that answers questions grounded in a provided document (PDF). The agent must use **Retrieval-Augmented Generation (RAG)**: retrieve relevant context from the document and generate an answer **only from that context**.

The provided earnings deck is the document used for assessment:

https://www.adanienterprises.com/-/media/Project/Enterprises/Investors/Investor-Downloads/Results-Presentations/AEL_Earnings_Presentation_Q2-FY26.pdf

Your solution must work for similar PDFs (reports/decks/policies/contracts).

- Sample 1
- Sample 2

What you must deliver (working code)

A runnable project that supports:

A) Ingestion + Indexing

- Input: a local PDF path (e.g., `./doc.pdf`)
- Extract text per page (or best effort)
- Chunk text and store metadata including **page number**
- Build a retrieval index (vector, sparse, or hybrid)

B) Conversational Q&A (multi-turn)

- A **chat loop** where the user can ask multiple questions in sequence
- The agent should keep conversation history without letting the model hallucinate beyond retrieved context

C) Grounded Answers with Citations

Every answer must include:

- A short final answer
- **Citations** pointing to the document (at minimum: page numbers; ideally page + chunk id), e.g.:
  - `[p13]` or `[p13:c42]`
- If the answer is not present in retrieved context, respond:
  - **"Not found in the document."**

D) Retrieval Visibility (debug)

Show what was retrieved for each question:

- top-k chunk snippets + their citations (page/chunk)
- retrieval scores if available

Constraints / Rules

- You may use **any model** (OpenAI/Gemini/Claude/open-source) and any vector DB (FAISS, Chroma, Pinecone, pgvector, Elasticsearch, etc.)
- Keep it runnable with a single command (e.g., `python main.py` or `node index.js`)
- No hardcoding answers specific to the sample document.
- Use Python, JavaScript or TypeScript as programming language.

Must-pass acceptance tests (we will run these)

Using the provided PDF:

1. **Grounded fact question**

- Q: "What are the major business segments discussed in the document?"
- Expected: Answer mentions segments present in the PDF and includes citations.

2. **Numeric question**

- Q: "What is the consolidated total income in H1-26?"
- Expected: Returns the correct value found in the PDF with citation(s). If unable to parse, must say "Not found in the document." (No guessing.)

3. **Cross-section question**

- Q: "What drivers are mentioned for EBITDA changes in H1-26?"
- Expected: Answer must reference the document's stated drivers and cite them.

4. **Negative control**

- Q: "What is the CEO's email address?"
- Expected: **"Not found in the document."**

5. **Conversational follow-up**

- Q1: "Summarize airport performance in H1-26."
- Q2: "Break that down into passenger and cargo changes."

- Expected: Q2 should be answered as a follow-up (chat history aware) and still cite the document.

What we evaluate (scoring)

Minimum (Pass)

- Runs end-to-end
- Retrieves relevant text
- Answers with citations
- No hallucination (correct “Not found” behaviour)
- Multi-turn chat loop works

Strong

- Robust chunking and metadata
- Good retrieval quality (top-k is relevant)
- Answer format is clean: “Answer + citations + evidence”
- Handles numeric questions without fabricating

Excellent (bonus)

- Hybrid retrieval (BM25 + vectors) or reranking
- Better PDF handling (tables, layout, page headers)
- Per-sentence citations or evidence mapping
- Lightweight evaluation: confidence thresholding / refusal logic